Desarrollo de un frontend React para el consumo de una API REST bancaria en Spring Boot

Maria Camila Castañeda Piedrahita

CC 1021805193

Universidad de Antioquia

Facultad de Ingeniería

Arquitectura de Software

Profesor: Diego José Luis Botia Valderrama

Septiembre de 2026

Medellín

Introducción

El desarrollo de aplicaciones web modernas requiere integrar de forma coherente la interfaz de usuario, la lógica de negocio y la persistencia de datos. En el Laboratorio N.º 1 de Arquitectura de Software se propuso completar una aplicación bancaria compuesta por una API REST desarrollada en Spring Boot y una base de datos MySQL. El ejercicio obligatorio consistió en construir un frontend básico capaz de consumir las rutas de los controladores para consultar clientes, realizar transferencias entre cuentas y visualizar el historial de transacciones asociado a cada cliente (Botia Valderrama, 2026).

Para atender el requerimiento se desarrolló una aplicación de página única con React y Vite. La interfaz organiza las operaciones en vistas independientes, utiliza React Router para la navegación y concentra el acceso a la API en un servicio basado en Axios. Esta separación evita distribuir las direcciones de los endpoints por todos los componentes y facilita el mantenimiento. En el entorno de desarrollo, el proxy de Vite redirige las solicitudes que comienzan con /api hacia el backend disponible en el puerto 8080.

Además de las tres vistas mínimas, se implementaron las funcionalidades opcionales indicadas en la guía: creación, actualización y eliminación de clientes. Para soportarlas de extremo a extremo se extendió el backend base con rutas PUT y DELETE, junto con los métodos correspondientes en la capa de servicio. El resultado es una solución integrada que conserva la arquitectura por capas propuesta por el docente y ofrece una interfaz clara para ejecutar las operaciones bancarias principales.

Este informe presenta los objetivos, las herramientas empleadas, la arquitectura general y el procedimiento de construcción e integración. También documenta las rutas consumidas, las validaciones aplicadas y la forma en que el frontend refleja los estados de carga, éxito, ausencia de datos y error. El código fuente completo se encuentra anexado mediante el repositorio de GitHub de la entrega (Castañeda Piedrahita, 2026).

Objetivos

Objetivo general

Desarrollar e integrar un frontend web en React con la API REST bancaria implementada en Spring Boot, de modo que el usuario pueda gestionar clientes, realizar transferencias entre cuentas y consultar el historial de movimientos mediante una interfaz organizada, comprensible y conectada con la persistencia en MySQL.

Objetivos específicos

- Analizar los controladores, DTO y servicios del backend suministrado para identificar las rutas, los métodos HTTP y la estructura de los datos requeridos por el frontend.
- Construir las vistas de consulta de clientes, transferencia de dinero e historial de transacciones por número de cuenta, de acuerdo con los requisitos obligatorios del laboratorio.
- Centralizar el consumo de la API mediante Axios y configurar un proxy de desarrollo en Vite que permita comunicar el frontend con Spring Boot a través de la ruta /api.
- Incorporar validaciones y mensajes de estado que orienten al usuario ante datos incompletos, montos inválidos, saldo insuficiente, operaciones exitosas o fallas de conexión.
- Implementar las funciones adicionales de crear, actualizar y eliminar clientes, incluyendo los endpoints y métodos de servicio necesarios en el backend.
- Documentar la instalación, configuración, ejecución y uso del sistema para facilitar su revisión, reproducción y mantenimiento.

Herramientas de software empleadas

La solución combina tecnologías de frontend, backend y persistencia. La Tabla 1 resume el propósito de cada herramienta dentro del proyecto. Las versiones corresponden a los archivos package.json y pom.xml incluidos en el repositorio de la entrega.

Tabla 1

Herramientas y tecnologías empleadas

Herramienta	Versión o componente	Uso en el proyecto
Java	17	Lenguaje de implementación del backend.
Spring Boot	3.5.11	Construcción y ejecución de la API REST.
Spring Web	Dependencia Maven	Definición de controladores y rutas HTTP.
Spring Data JPA / Hibernate	Dependencia Maven	Acceso a datos y mapeo objeto-relacional.

Herramienta	Versión o componente	Uso en el proyecto
MapStruct	1.5.5.Final	Conversión entre entidades y DTO.
Lombok	1.18.30	Reducción de código repetitivo en clases Java.
MySQL Connector/J	Dependencia Maven	Conexión JDBC con MySQL.
MySQL	Servidor local	Persistencia de clientes y transacciones.
React / React DOM	19	Construcción de la interfaz por componentes.
React Router DOM	7.18.3	Navegación entre vistas sin recargar la aplicación.
Axios	1.20.0	Envío de solicitudes HTTP y manejo de respuestas.
Vite	8.2.2	Servidor de desarrollo, proxy y compilación del frontend.
Tailwind CSS	4.3.3	Dependencia disponible para estilos del proyecto.
Lucide React	1.16.0	Iconos de navegación, acciones y estados.
Maven Wrapper	Incluido	Gestión reproducible de dependencias y ejecución del backend.
Git y GitHub	Repositorio remoto	Control de versiones, documentación y entrega.
Visual Studio Code / navegador	Entorno de desarrollo	Edición, ejecución y comprobación visual del frontend.

React permite construir interfaces declarativas basadas en componentes reutilizables (Meta Open Source, s. f.), mientras que Axios proporciona una API basada en promesas para realizar solicitudes HTTP (Axios, s. f.). En el backend, Spring Boot simplifica la configuración de aplicaciones Java y su integración con tecnologías de persistencia (Broadcom, s. f.). MySQL actúa como sistema gestor de la base de datos relacional (Oracle, s. f.).

Arquitectura propuesta

La aplicación adopta una arquitectura cliente-servidor y conserva la organización por capas definida para el backend del laboratorio. El frontend funciona como cliente web, genera solicitudes HTTP y procesa respuestas JSON. El backend expone los recursos mediante controladores REST, ejecuta las reglas de negocio en servicios y accede a MySQL a través de repositorios JPA. La separación de responsabilidades reduce el acoplamiento entre la presentación, la lógica y la persistencia, principio asociado a las arquitecturas por capas (Fowler, 2002).

Figura 1

Arquitectura general de la solución



Flujo bidireccional de solicitudes HTTP, respuestas JSON y persistencia

Nota. Elaboración propia a partir de la arquitectura del backend y del código del frontend.

Capa de presentación

La capa de presentación está implementada con React. App.jsx define un BrowserRouter y cinco rutas: inicio, clientes, creación de cliente, transferencia e historial. Las páginas se apoyan en componentes comunes para la barra de navegación, los encabezados, las alertas, los indicadores de carga, los estados vacíos y las ventanas modales. Esta reutilización mantiene una experiencia consistente y evita repetir estructuras visuales.

Cada página administra su propio estado con useState y ejecuta las consultas necesarias con useEffect. La vista de clientes presenta una tabla con saldo y número de cuenta, y ofrece acciones para consultar el detalle, abrir el historial, iniciar una transferencia, editar o eliminar. La vista de transferencia selecciona cuentas existentes y valida el monto antes de enviarlo. La vista de historial permite escoger una cuenta, mostrar su saldo y clasificar cada movimiento como recibido o enviado.

Capa de integración

El archivo src/services/api.js crea una instancia de Axios con /api como URL base y expone funciones específicas para cada operación. Esta decisión centraliza la comunicación y permite que las páginas se concentren en la interacción del usuario. En modo de desarrollo, vite.config.js redirige /api a http://localhost:8080. Así, el navegador se comunica con el servidor de Vite y este reenvía las solicitudes al backend, evitando incorporar direcciones absolutas en cada componente.

Capas del backend y persistencia

Los controladores reciben las solicitudes y delegan el procesamiento a los servicios. CustomerService administra las operaciones sobre clientes y TransactionService valida las cuentas, verifica el saldo, actualiza los balances y registra cada transferencia. Los DTO controlan la información transferida entre capas; los mappers convierten entre DTO y entidad; y los repositorios extienden JpaRepository para proporcionar las operaciones de persistencia. Hibernate actualiza el esquema configurado para la base lab12026p mediante spring.jpa.hibernate.ddl-auto=update.

Rutas integradas

Tabla 2

Endpoints consumidos por el frontend

Método	Ruta	Función	Tipo
GET	/api/customers	Listar todos los clientes.	Obligatoria
GET	/api/customers/{id}	Consultar el detalle de un cliente.	Complementaria
POST	/api/customers	Crear un cliente.	Adicional en el frontend; provista por el backend base
PUT	/api/customers/{id}	Actualizar un cliente.	Opcional
DELETE	/api/customers/{id}	Eliminar un cliente.	Opcional
POST	/api/transactions	Realizar y registrar una transferencia.	Obligatoria
GET	/api/transactions/{accountNumber}	Consultar el historial de una cuenta.	Obligatoria

El flujo principal inicia cuando una página invoca una función del servicio Axios. La solicitud atraviesa el proxy y llega al controlador correspondiente. Luego el servicio aplica las validaciones y utiliza el repositorio para consultar o modificar la base de datos. La respuesta se convierte a JSON y regresa al frontend, donde se actualiza el estado de la página y se presenta el resultado al usuario.

Procedimiento

1. Revisión de requisitos y backend base

Se estudió la guía del laboratorio y el repositorio base del docente para identificar las entidades Customer y Transaction, los DTO, los servicios y los controladores existentes. La revisión permitió establecer los datos que debía enviar o recibir cada vista: nombres, apellidos, número

de cuenta y saldo para clientes; y cuenta de origen, cuenta de destino, monto y fecha para transacciones.

2. Preparación de la base de datos y el backend

Se configuró una base de datos MySQL denominada lab12026p. La URL JDBC, el puerto, la base de datos y el usuario se conservaron en application.properties, mientras que la contraseña se externalizó mediante la propiedad spring.datasource.password=${DB_PASSWORD}. Antes de ejecutar el backend se definió DB_PASSWORD únicamente en la terminal de PowerShell. El backend se mantuvo en el puerto 8080 y se conservó la actualización automática del esquema con Hibernate. Para ejecutar el servidor se utilizó Maven Wrapper desde la carpeta lab12026p mediante el comando .\mvnw.cmd spring-boot:run.

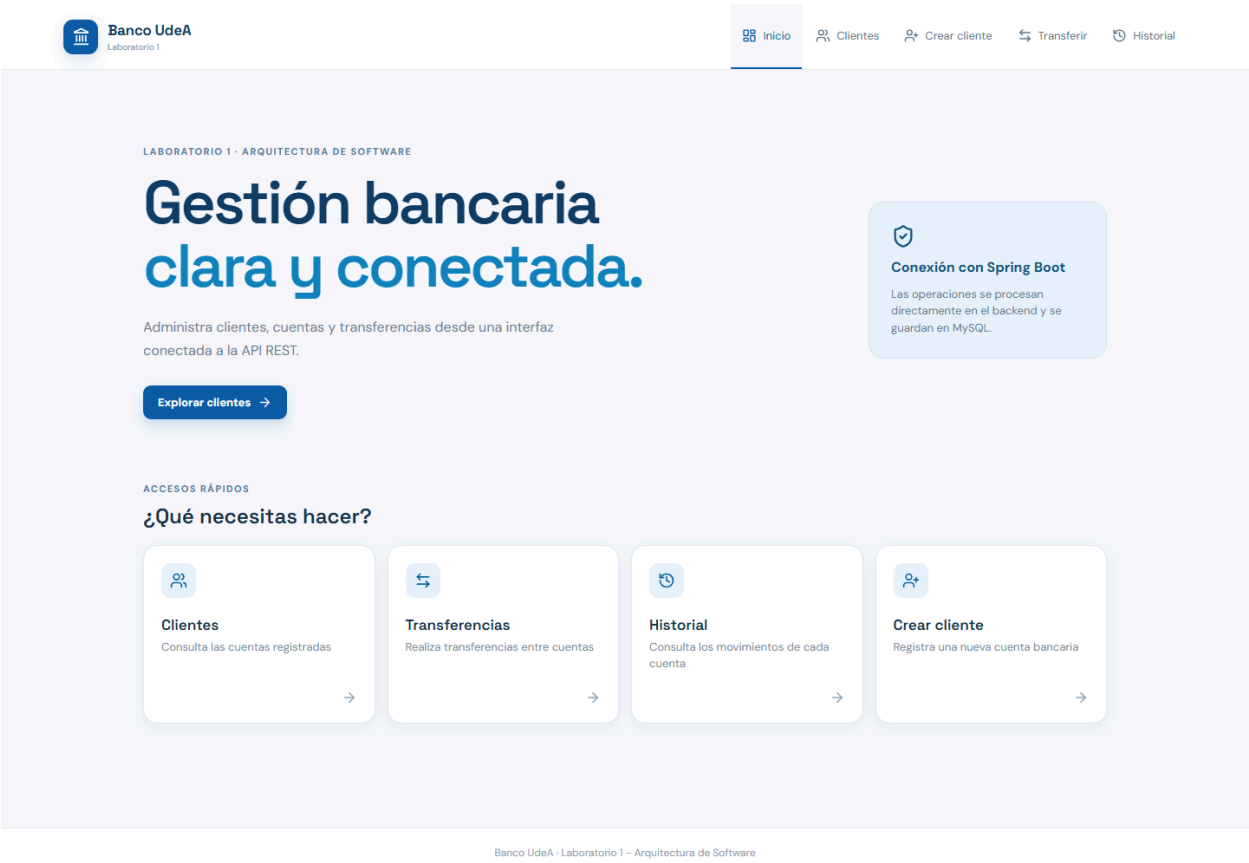
Antes de iniciar el frontend se verificó que MySQL estuviera activo y que Spring Boot pudiera establecer la conexión. Este orden es necesario porque las consultas y transferencias dependen de los datos persistidos; si el backend no está disponible, el frontend muestra un mensaje que orienta a revisar la ejecución de Spring Boot.

3. Creación y organización del frontend

El frontend se organizó en las carpetas src/pages, src/services y src/utils. App.jsx concentra las rutas, components.jsx reúne componentes reutilizables y main.jsx monta la aplicación. La hoja index.css define una interfaz adaptable a pantallas de escritorio y móviles mediante cuadrículas, tablas con desplazamiento horizontal y reglas responsive.

Figura 2

Pantalla de inicio y navegación principal



Nota. Captura elaborada a partir de la ejecución local del proyecto.

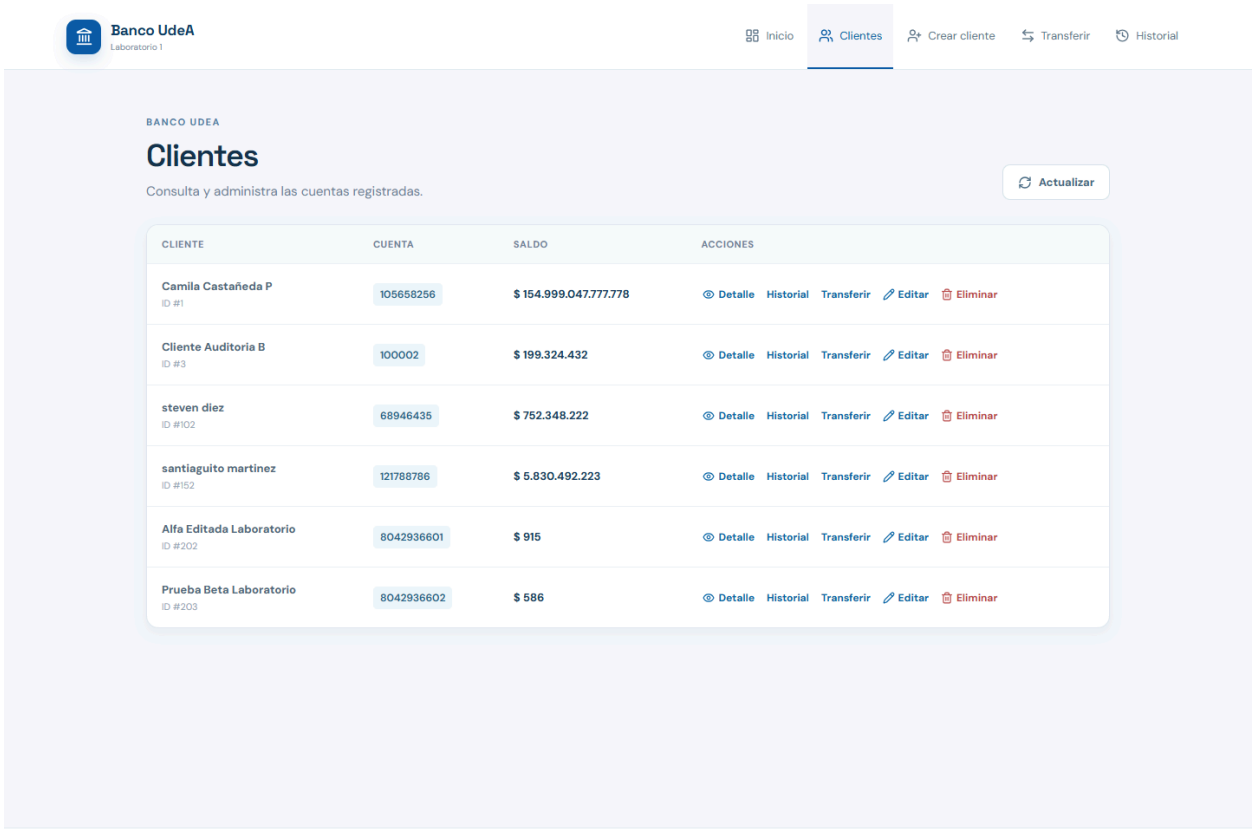
4. Implementación de la consulta y gestión de clientes

CustomersPage.jsx solicita la lista con GET /api/customers y representa los resultados en una tabla. Mientras se obtiene la respuesta muestra un estado de carga; si no existen registros presenta un estado vacío; y si ocurre una falla, utiliza getApiError para traducirla en un mensaje comprensible. La acción Detalle consulta GET /api/customers/{id} y presenta la información dentro de una ventana modal.

Como ampliación del laboratorio, la misma vista incorpora edición y eliminación. La edición precarga los datos y envía PUT /api/customers/{id}; la eliminación solicita confirmación y ejecuta DELETE /api/customers/{id}. Después de una operación exitosa se vuelve a consultar la lista para sincronizar la interfaz con la base de datos.

Figura 3

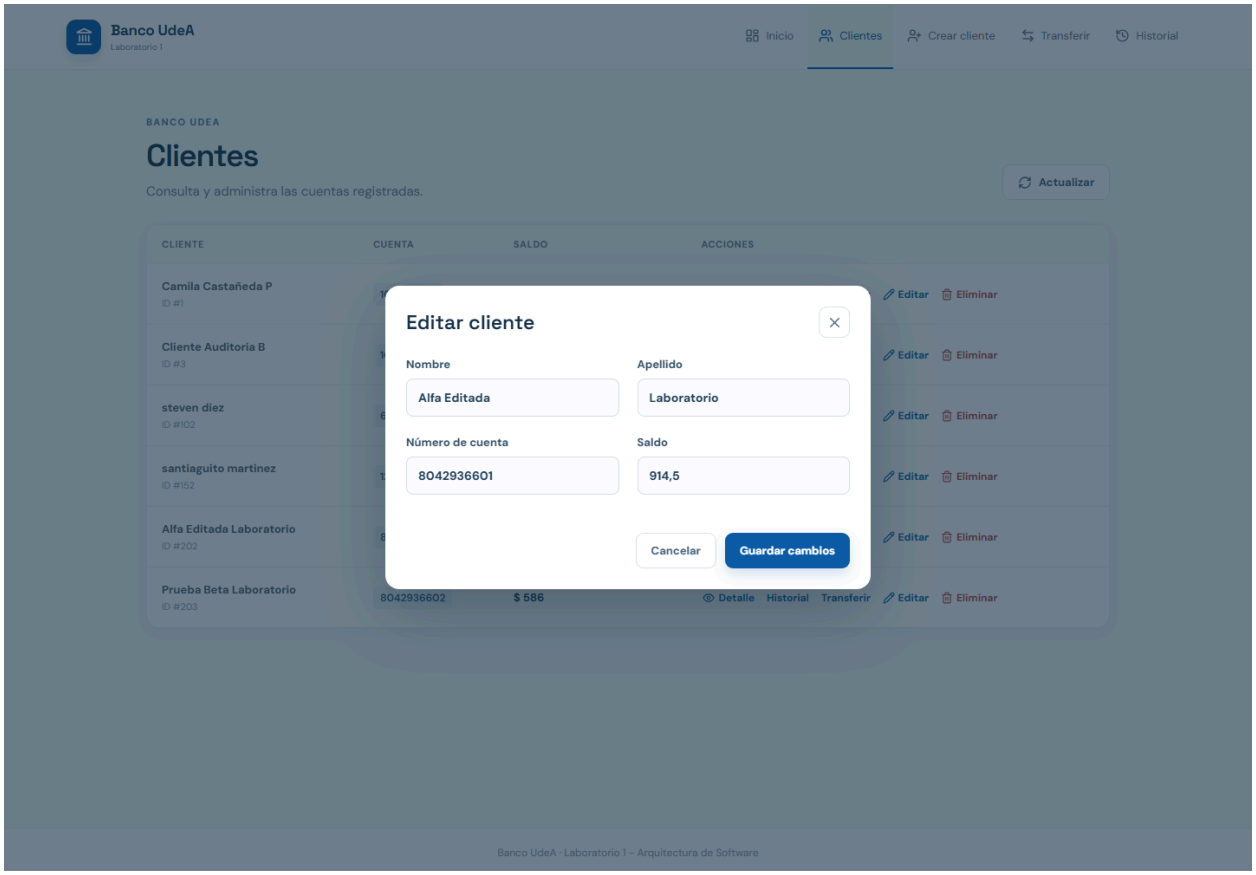
Consulta de clientes y acciones adicionales



Nota. Captura elaborada a partir de la ejecución local del proyecto.

Figura 4

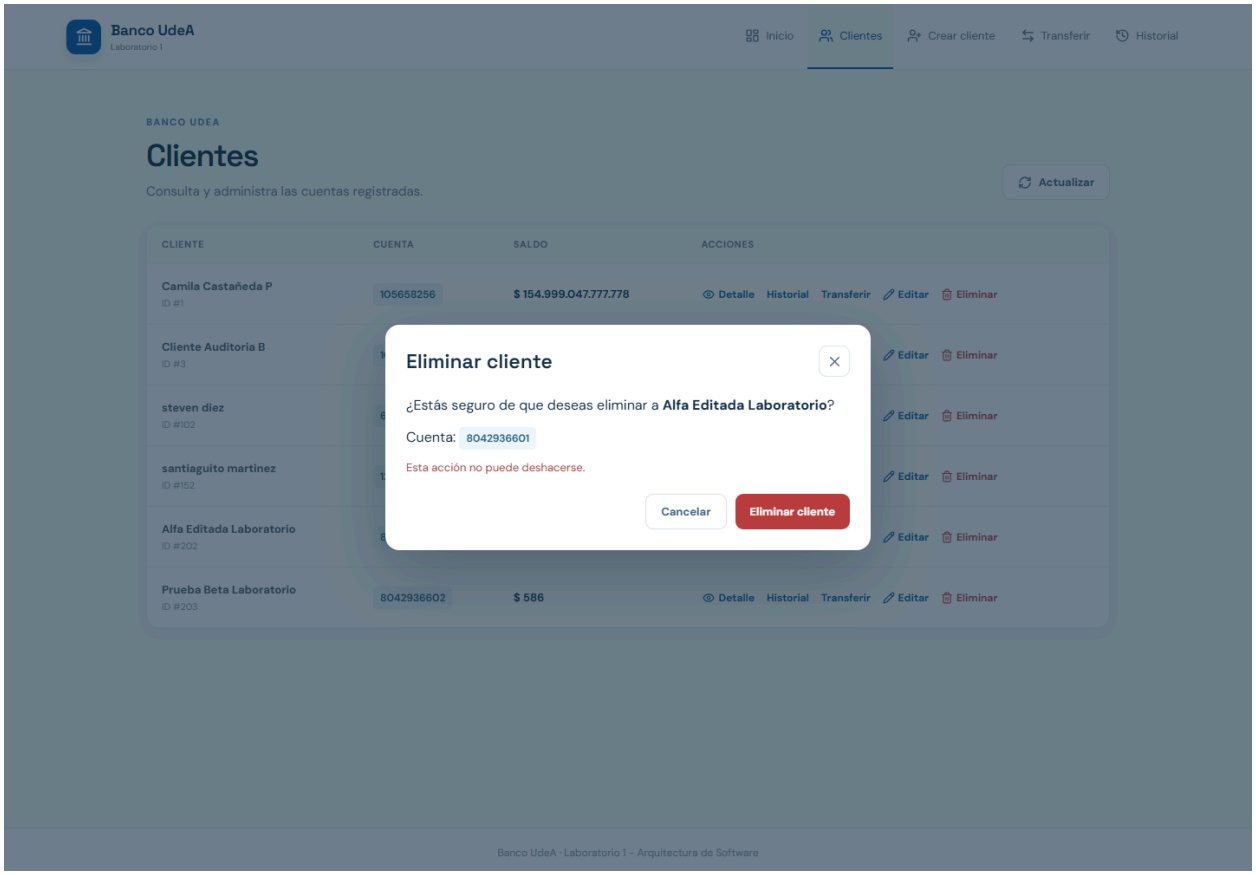
Formulario de edición de un cliente



Nota. Captura elaborada a partir de la ejecución local del proyecto.

Figura 5

Confirmación para eliminar un cliente



Nota. Captura elaborada a partir de la ejecución local del proyecto.

5. Implementación de creación de clientes

CreateCustomerPage.jsx contiene un formulario controlado para nombre, apellido, número de cuenta y saldo inicial. Antes de invocar POST /api/customers verifica que todos los campos estén completos, que el saldo sea numérico y que no sea negativo. Cuando el registro se completa, limpia el formulario y muestra una confirmación. La vista ofrece además un acceso directo a la tabla de clientes.

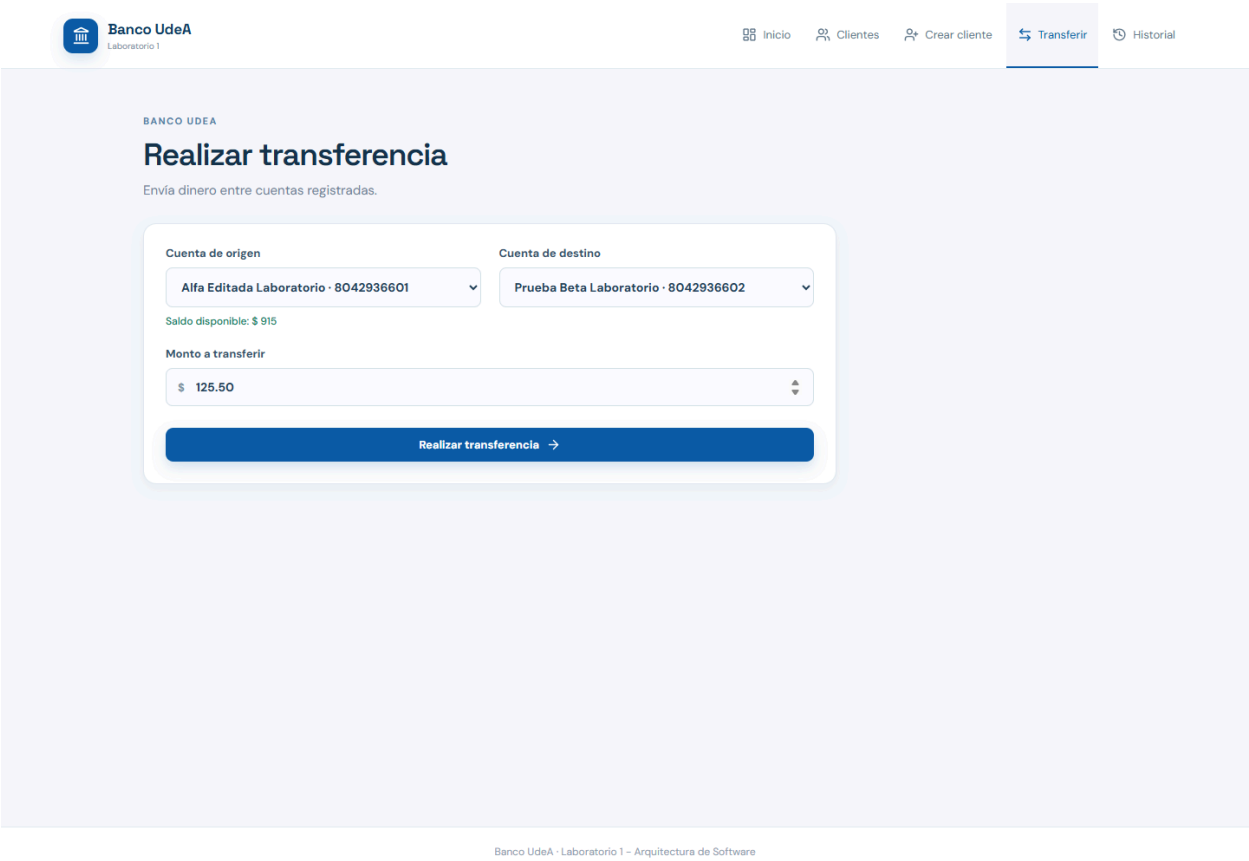
6. Implementación de transferencias

TransferPage.jsx carga las cuentas disponibles para evitar que el usuario escriba números inexistentes. El formulario impide seleccionar la misma cuenta como origen y destino, exige un monto mayor que cero y comprueba en la interfaz que el valor no supere el saldo disponible. La solicitud POST /api/transactions se envía únicamente después de estas verificaciones. En el

backend, TransactionService comprueba que los números de cuenta no sean nulos, que ambas cuentas existan y que el remitente tenga saldo suficiente; después actualiza ambos saldos y registra el movimiento. Las verificaciones de cuentas diferentes y monto positivo se realizan en el frontend, pero no están repetidas actualmente en el servicio del backend.

Figura 6

Formulario para realizar una transferencia



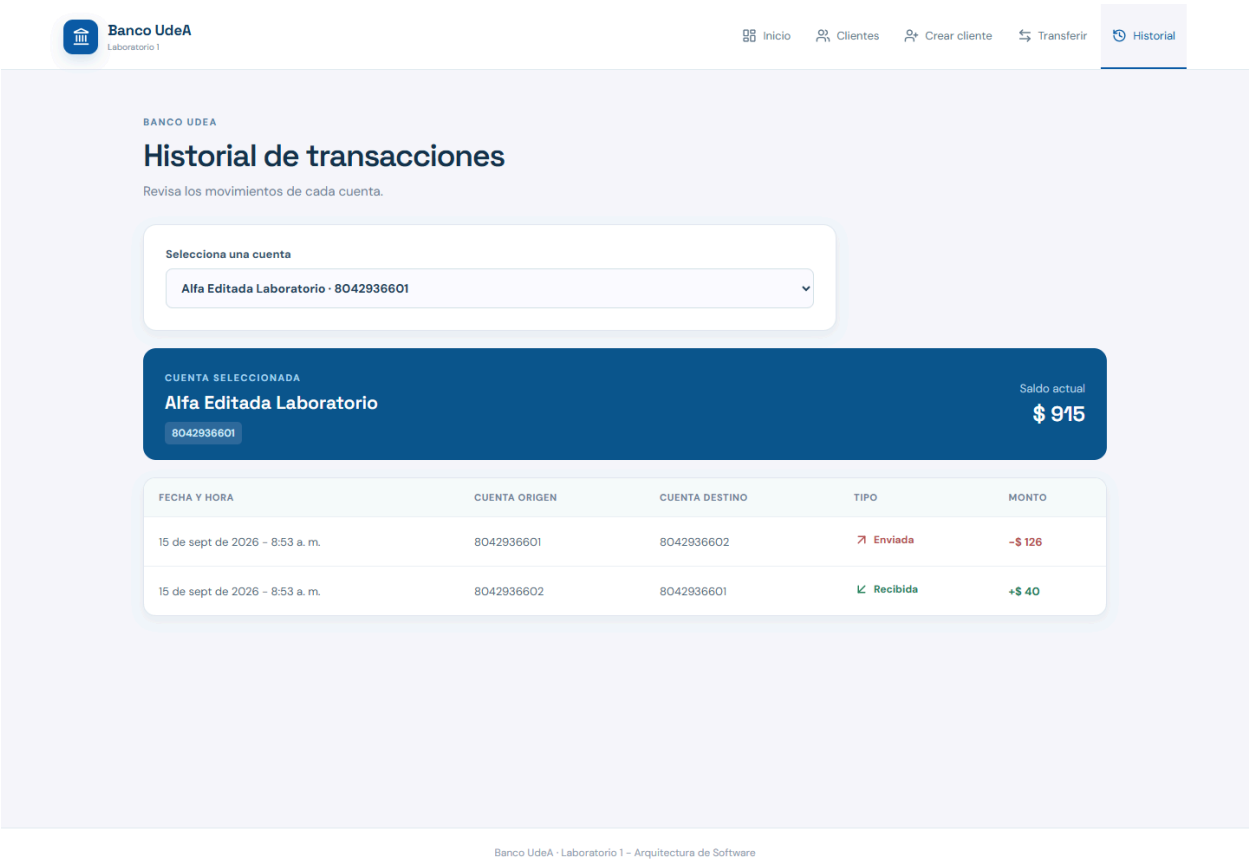
Nota. Captura elaborada a partir de la ejecución local del proyecto.

7. Implementación del historial por cuenta

HistoryPage.jsx consulta primero los clientes para construir el selector de cuentas. Cuando se elige una opción, solicita GET /api/transactions/{accountNumber}. La tabla presenta fecha y hora, cuenta origen, cuenta destino, tipo de movimiento y monto. La interfaz determina si cada transacción fue recibida o enviada comparando la cuenta seleccionada con receiverAccountNumber y ajusta el signo del monto para facilitar la lectura.

Figura 7

Historial de movimientos de una cuenta



Nota. Captura elaborada a partir de la ejecución local del proyecto.

8. Extensión del backend para las funciones opcionales

El backend del docente fue ampliado en CustomerController con los métodos updateCustomer y deleteCustomer, expuestos mediante PUT y DELETE. En CustomerService, la actualización busca el cliente, verifica que el nuevo número de cuenta no pertenezca a otro registro, modifica los atributos y guarda la entidad. La eliminación comprueba la existencia del cliente antes de llamar al repositorio. Los errores de negocio se devuelven como respuestas de solicitud incorrecta con mensajes que el frontend puede presentar.

9. Integración, validaciones y comprobación

La comprobación funcional se realizó con MySQL, Spring Boot y Vite ejecutándose conjuntamente. Desde la interfaz real se consultaron los clientes, se crearon registros de prueba, se abrió el detalle, se actualizó un cliente y se eliminó un registro temporal. También se

verificaron el rechazo de cuentas iguales, de un monto no válido y de un monto superior al saldo, además de dos transferencias entre cuentas diferentes. Las consultas posteriores confirmaron la actualización de los saldos, el registro de las transacciones y la clasificación de movimientos recibidos y enviados en el historial.

Tabla 3

Casos de comprobación de la integración

Caso	**Acción**	**Resultado esperado**
Consulta de clientes	Abrir /clientes con el backend activo.	La tabla presenta los registros guardados en MySQL.
Creación válida	Completar los cuatro campos y enviar.	El cliente se registra y aparece en la consulta.
Saldo inválido	Ingresar un saldo negativo.	El frontend bloquea el envío y muestra la validación.
Transferencia válida	Elegir cuentas distintas y un monto disponible.	Se actualizan los saldos y se registra el movimiento.
Cuentas iguales	Seleccionar la misma cuenta en origen y destino.	El frontend impide la operación.
Saldo insuficiente	Ingresar un monto mayor al saldo.	La operación se rechaza con un mensaje claro.
Historial	Seleccionar una cuenta con movimientos.	La tabla clasifica las transacciones recibidas y enviadas.
Actualización	Editar los datos de un cliente.	Los cambios se persisten y la lista se actualiza.
Eliminación	Confirmar el borrado de un cliente de prueba.	El backend responde sin contenido y el registro desaparece.
Backend inactivo	Abrir una vista que consulta /api.	Se informa que no fue posible conectar con Spring Boot.

Como verificación técnica adicional, el frontend se instaló con npm ci y se compiló con npm run build. Vite 8.2.2 procesó 1.916 módulos y generó correctamente los archivos de distribución, sin errores de compilación. El backend se inició con Maven Wrapper utilizando DB_PASSWORD, estableció la conexión con la base de datos MySQL lab12026p y respondió correctamente a GET /api/customers. Vite sirvió la aplicación en el puerto 5173 y el proxy /api permitió comprobar la comunicación real entre Axios y Spring Boot.

10. Publicación y documentación

El proyecto se organizó en un único repositorio con dos directorios principales: lab12026p para Spring Boot y lab12026p-frontend para React/Vite. El README describe la arquitectura, las tecnologías, los endpoints, los requisitos, la configuración de MySQL, los comandos de ejecución, el orden de inicio y un recorrido básico de uso. Esta documentación permite que otra persona clone el repositorio y reproduzca la solución con menor dependencia de explicaciones adicionales.

Conclusiones

La integración desarrollada cumple los tres requisitos obligatorios del laboratorio: consultar clientes, realizar transferencias entre cuentas y visualizar el historial de transacciones por cliente. Las vistas no operan de forma aislada; consumen las rutas definidas en los controladores y reflejan los datos administrados por el backend.

La separación entre páginas, componentes reutilizables y servicio de API favorece el mantenimiento del frontend. Centralizar las solicitudes en api.js y utilizar el proxy de Vite evita duplicar configuraciones y permite modificar el destino del backend sin alterar cada pantalla.

La arquitectura por capas del backend mantiene responsabilidades diferenciadas. Los controladores atienden las solicitudes, los servicios aplican la lógica de negocio, los DTO y mappers controlan la transferencia de información, y los repositorios gestionan la persistencia mediante JPA e Hibernate.

Las validaciones implementadas en el frontend mejoran la experiencia del usuario al detectar datos incompletos, cuentas iguales, montos no positivos y saldos insuficientes antes de enviar la solicitud. El backend valida la presencia y existencia de las cuentas y el saldo disponible; sin embargo, no repite actualmente las comprobaciones de cuentas diferentes ni de monto positivo, por lo que un consumidor directo de la API podría omitir esas dos reglas.

Las funcionalidades opcionales amplían la propuesta desde una consulta básica hacia una gestión más completa de clientes. La incorporación de PUT y DELETE demuestra que la

interfaz y el backend pueden evolucionar de manera coordinada sin modificar la estructura general de la solución.

El uso de GitHub y de un README detallado complementa la implementación técnica, pues conserva el historial del código y facilita la instalación, ejecución y evaluación del proyecto.

Bibliografía

Axios. (s. f.). Getting started. https://axios-http.com/docs/intro

Botia Valderrama, D. J. L. (2026). Laboratorio N.º 1: Introducción a Spring Boot https://drive.google.com/file/d/1TRjJXKZXq3nv-I-p4GBTKYboMF6kzt2L/view Universidad de Antioquia.

Broadcom. (s. f.). Spring Boot reference documentation. https://docs.spring.io/spring-boot/reference/

Castañeda Piedrahita, M. C. (2026). lab12026 [Código fuente]. GitHub. https://github.com/Camii1234/lab12026

Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley.

Meta Open Source. (s. f.). React documentation. https://react.dev/

Oracle. (s. f.). MySQL reference manual. https://dev.mysql.com/doc/

Vite. (s. f.). Getting started. https://vite.dev/guide/

Proyecto anexo en GitHub

El código fuente del frontend, las extensiones realizadas al backend y la documentación de instalación se encuentran en el siguiente repositorio:
https://github.com/Camii1234/lab12026

Estructura principal del anexo:

- lab12026p/: backend Spring Boot, controladores, servicios, repositorios, entidades, DTO, mappers y configuración de MySQL.
- lab12026p-frontend/: frontend React/Vite, páginas, componentes, servicio Axios, estilos y archivos de compilación.
- README.md: descripción, arquitectura, tecnologías, endpoints, requisitos, configuración y pasos de ejecución.